



Neural Ordinary Differential Equations

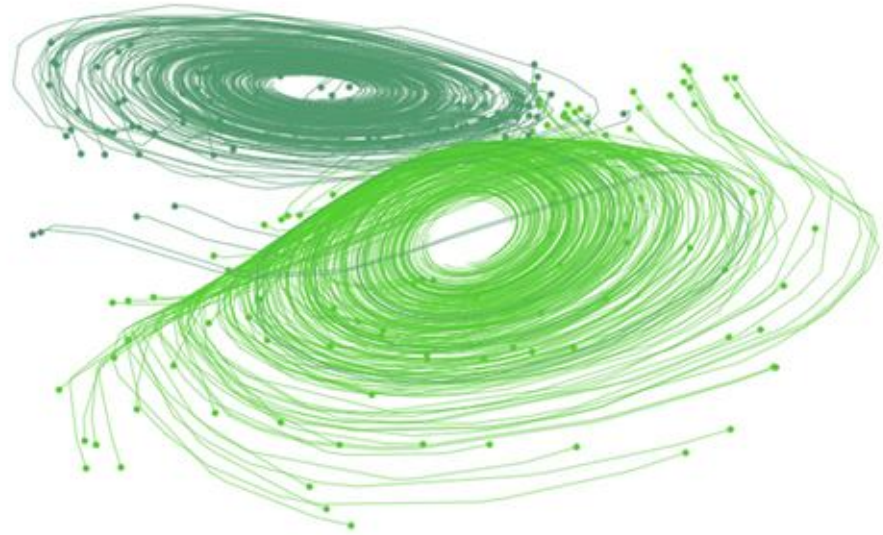
Rammini Nipuni Senani de Silva

Department of Mathematics
Clarkson University, Potsdam
desilvr@clarkson.edu

March 7, 2023

Content

- ODEs and Solving ODEs
- Neural ODEs
- Neural ODE Simulations



Ordinary Differential Equations

$$\frac{dy}{dt} = f(y, t) \quad \begin{cases} \rightarrow \text{Analytical Solutions} \\ \rightarrow \text{Numerical Solutions} \end{cases}$$

- **Analytical Solutions**

$$\frac{dv}{dt} + av = b \quad \longrightarrow \quad v(t) = (b/a) + ce^{-at} \quad \text{where } c \text{ is constant.}$$

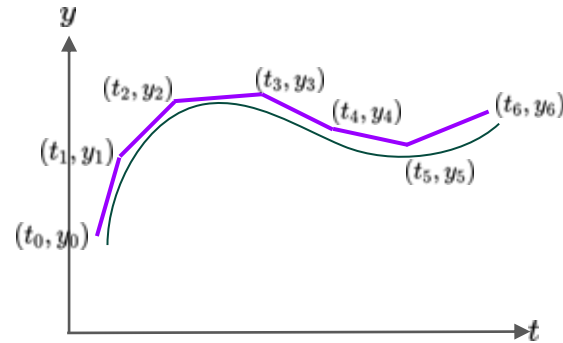
- **Numerical Solutions**

$$\frac{dy}{dt} = f(y, t); \quad y(t_0) = y_0$$

By using a numerical solver

Eg: Euler Method

$$y_{i+1} = y_i + f(t_i, y_i)(t_{i+1} - t_i)$$



Neural ODEs

Neural Ordinary Differential Equations

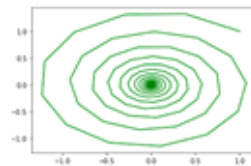
- Given system dynamics can be described using ODEs
- We don't know what the ODE model $f(y, t)$ looks like

$$y' = \underbrace{f(y, t)}_{\text{unknown}}$$

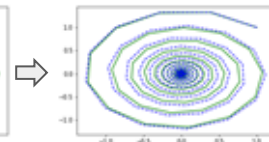
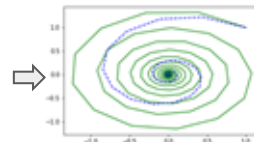
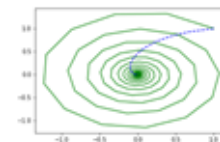
- We have time series data which fit to this ODE model

We create a Neural Network to obtain the ODE model $f(y, t)$ and use data to determine its parameters.

$$f(y, t) \Rightarrow f(h(t), t, \theta) \longrightarrow \text{Neural ODE model}$$



$$y' = \begin{bmatrix} -0.1 & 2.0 \\ -2.0 & -0.1 \end{bmatrix} y$$



Neural Ordinary Differential Equations

$$\frac{dy}{dt} = f(y, t) \quad ; \quad y(t_0) = y_0$$

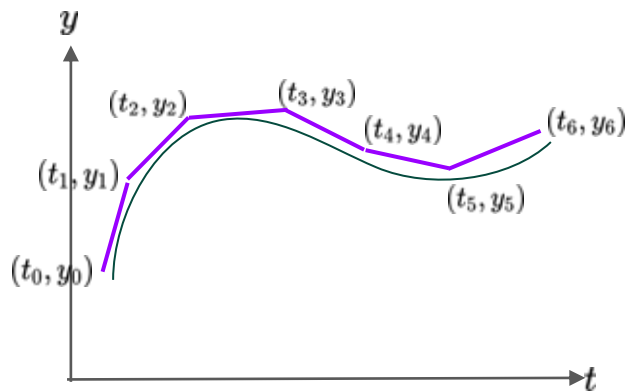
Neural ODEs are class of models that parametrize the rate of change $\dot{y}$ of the state y with the help of Neural network .

$$\dot{y} = f_{\theta}(y, t)$$

NODEs predict the $\tilde{y}_i^T$ at t_i^T such that

$$y(t_i^T) = y_0 + \int_{t_0}^{t_i^T} f_{\theta}(y(t), t) dt$$

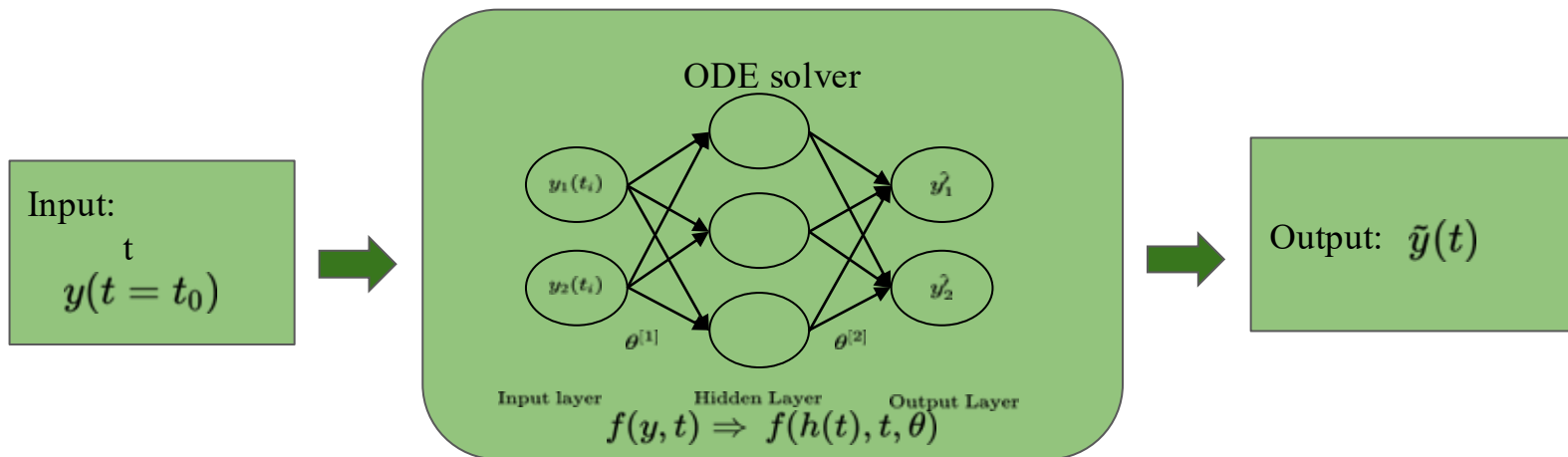
when IC (t_0, y_0) given.



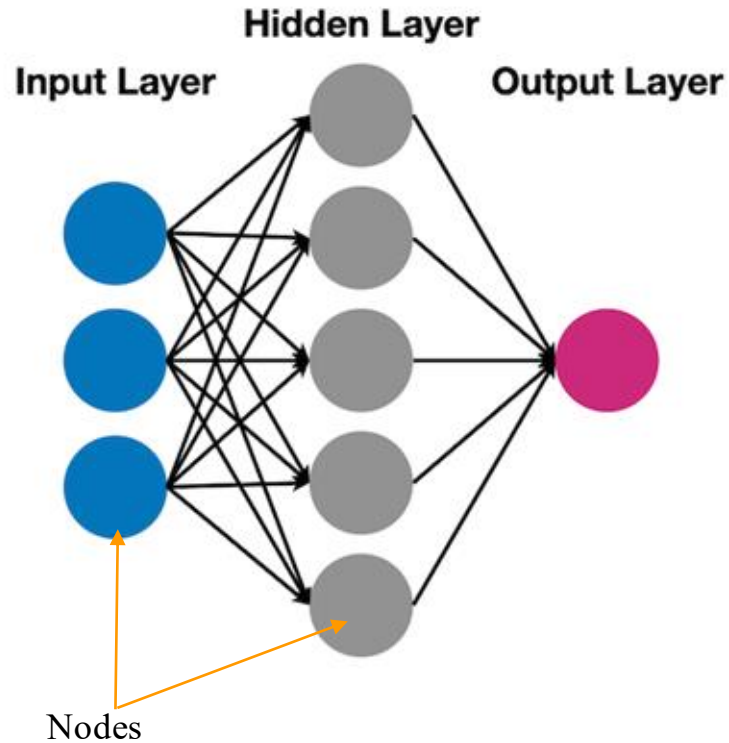
Neural ODEs: Implementation

- Let the ODEs right hand side be a Neural Network.

$$f(y, t) \Rightarrow f(h(t), t, \theta)$$

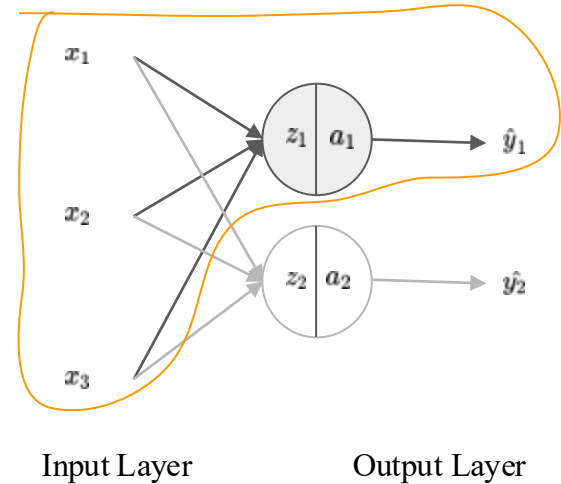


Neural Network Architecture

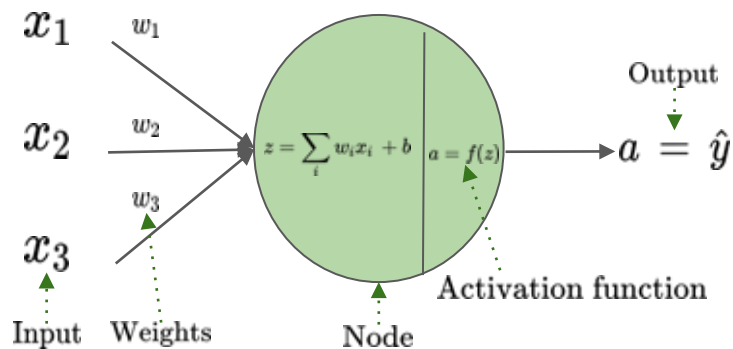


This neural network gives two dimensional output (y) to three dimensional input (x)

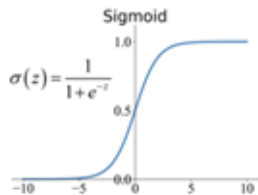
$$NN : x \longrightarrow y$$



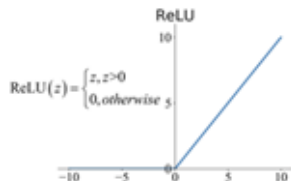
Neural Network Representation



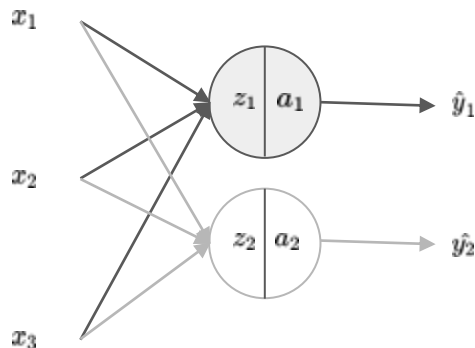
f = Activation Function
Ex: Sigmoid



Rectified



- For one input (column vector)



$$z_{(2,1)} = W_{(2,3)}^T x_{(3,1)} + b_{(2,1)}$$

$$\hat{y}_{(2,1)} = \sigma(z_{(2,1)})$$

- For m inputs

$$Z_{(2,m)} = W_{(2,3)}^T X_{(3,m)} + b_{(2,m)}$$

Training a Neural Network

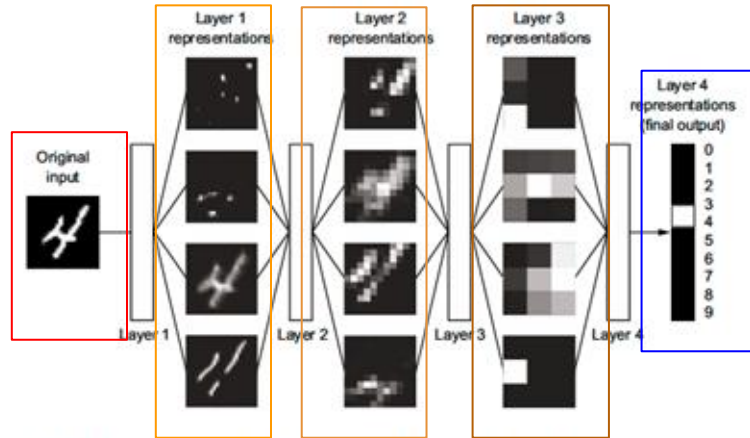
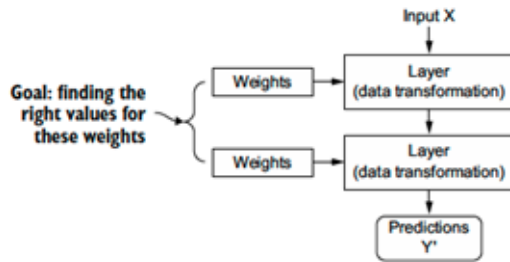


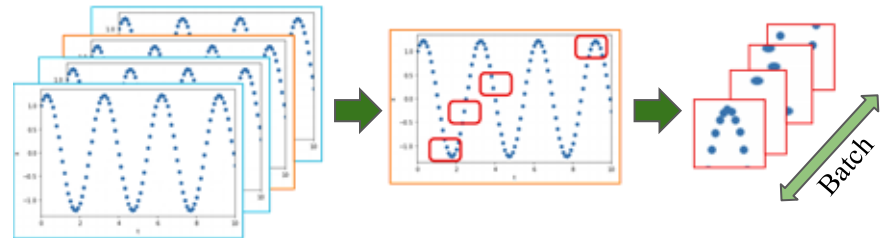
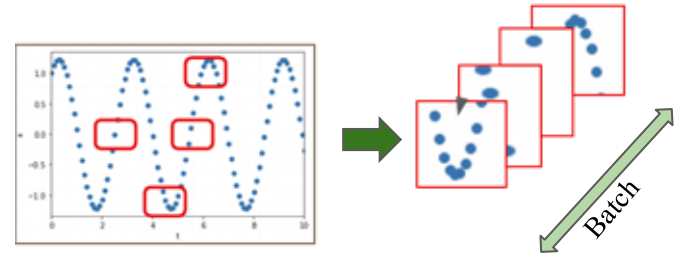
Figure 1.6 Data representations learned by a digit-classification model



ODE : Training NODE

Data

- We can train the Neural ODE model by taking one trajectory(one IC) of the system .
 - Train data:
for every training iteration, sequence of observed states of the trajectory at different points in time.
 - Validation data, Test data:
full trajectory generated by random IC.
- We can train the Neural ODE model by taking multiple trajectories of the system.
 - Train data:
for every training iteration, sequence of observed states of a random trajectory at different points in time.
 - Validation data, Test data:
trajectory generated by random IC.



Neural ODE : Training the model

Algorithm:

Initialization: Setting up the Neural Network parameters with random weights

For each iteration from 1 to k

Forward pass:

Generate batch from train data

$\text{predY} = \text{ODESolver}(t, \theta)$, batchICs, batchTime,
solver)

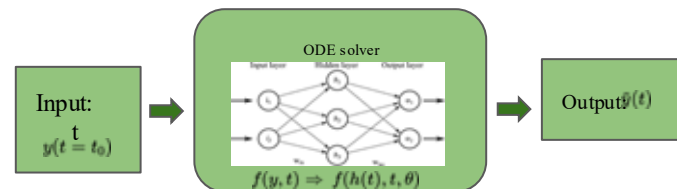
$$\text{Loss} = \text{Loss computation: } \sum_{\text{batch}} \frac{1}{N} \sum_{t=1}^N |\text{predY}(t) - \text{batchY}(t)|$$

Backward pass:

Backpropagation (Loss)

Parameter update:

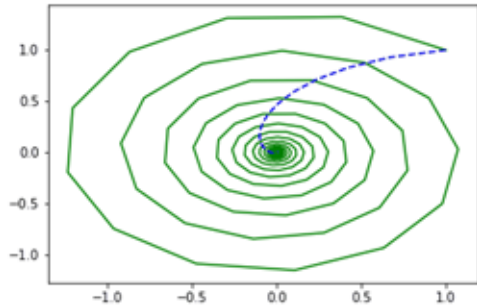
Optimize by minimizing Loss



Neural ODE Simulations

Spiral System

$$y' = \begin{bmatrix} -0.1 & 2.0 \\ -2.0 & -0.1 \end{bmatrix} y$$



Depth of Network layers: 2,50,2

Activation functions: Tanh

Train data: Trajectory generated from Initial Conditions [0, 2]

Train data : 2000

Optimizer :RMSProp with lr = 1e(-3)

Epochs: 5000

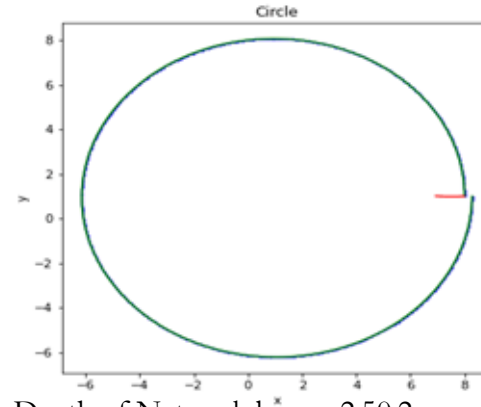
Batch size: 24

Batch time: 10

Test data: Full trajectory

Circle with center (a,b)

$$\begin{aligned} y'_1 &= -y_2 + b \\ y'_2 &= y_1 - a \end{aligned}$$



Depth of Network layers: 2,50,2

Activation functions: Tanh

Train data: Trajectory generated from Initial Conditions [8,1]

Train data : 500

Optimizer :RMSProp with lr = 1e(-3)

Epochs: 400

Batch size: 10

Batch time: 8

Test data: Full trajectory

Neural ODE Simulations

Van der Pol Oscillator

$$\frac{d^2y}{dt^2} - \mu(1 - y^2) \frac{dy}{dt} + y = 0$$

$$\dot{y}_1 = y_2$$

$$\dot{y}_2 = \mu(1 - y_1^2)y_2 - y_1$$

Depth of Network layers: 2,50,2

Activation functions: Tanh

Train data: Trajectories generated from 10 random Initial Conditions

Train data : [10, 200]

Optimizer :RMSProp with lr = 1e(-3)

Epochs: 5000

Batch size: 24

Batch time: 10

Test data: trajectory generated from random IC

